INSTITUTO SUPERIOR TECNOLÓGICO QUITO

Tercer Módulo – Python, Tipos de Datos - Data Types

python™

Carlos Alberto Pucha

Comentarios

¿Qué son?

Texto contenido en ficheros de Python que es ignorado por el interprete; es decir no es ejecutado.

¿Cúal es su utilidad?

- Se puede utilizar para documentar código y hacerlo más legible.
- Preferiblemente trataremos de hacer fácil de entender y que necesite pocos comentarios, en lugar de vernos forzados a recurrir a los comentarios para explicar el código.

Tipos de comentarios

Comentarios de una línea

Texto precedido por '#' Se suele usar para documentar expresiones sencillas

[1]:
```
# Esto es una instruccion print
print("Hello word") # Esto es una instruccion print
```

```
Hello word
```

Comentarios de varias línea

- Texto encapsulado en triple comillas (que pueden ser simples como dobles)
- Se suele usar para documentar bloques de código más significativo

[2]:
```
# creo una funcion de multiplicación
def producto(x, y):
    '''
    Esta función recibe dos números como parámetros y devuelve como resultado el producto de los mismos
     '''
    #creo una variable donde guardo el producto de los números recibidos
    c = x * y
    return c
```

[3]:
```
# Llamo a la función
producto(3, 7)
```

[3]: 21

Literales, variables y tipos de datos básicos

De forma muy genérica al ejecutarse un programa en Python se realizan *operaciones* sobre *objetos*

Estos dos términos son fundamentale.

Objetos: Cualquier tipo de datos (números, caracteres o datos más complejos).

- *Operaciones*: Cómo manipulamos estos datos. +,-,*,/

  Ejemplo:

[4]:
```
4 * 3
```

[4]:
```
12
```

Literales

- Python tienen una serie de tipos de datos integrados en el propio lenguaje.
- Los literales son expresiones que generan objetos de estos tipos.
- Estos objetos, según su tipo pueden ser
  - Simples o compuestos
  - Mutables o inmutables

Literales simples

- Enteros
- Decimales o punto flotante
- Booleanos

[5]:
```
print(4)              #número entero
print(4.3)            #número en coma flotante
print('Hello word!')  #objeto de tipo string
print(False)          #Objeto de tipo booleano
```

```
4
4.3
Hello word!
False
```

Literales compuestos

- Tuplas
- Listas
- Diccionarios
- Conjuntos

[6]:
```
print([1, 2, 3, 3]) # lista - mutable
print({'Nombre' : 'John Doe', "edad": 30}) # Diccionario - mutable
print({1, 2, 3, 3}) # Conjunto - mutable
print((4, 5)) # tupla - inmutable
2, 4 # tupla
```

```
[1, 2, 3, 3]
{'Nombre': 'John Doe', 'edad': 30}
{1, 2, 3}
(4, 5)
```

[6]:
```
(2, 4)
```

Variables

- Referencias a objetos.
- Las variables y los objetos se almacenan en diferentes zonas de memoria.
- Las variables siempre referencian a objetos y nunca a otras variables.
- Objetos sí que pueden referenciar a otros objetos. Ejemplo: listas.

- Sentencia de asignación:

'='

[7]:
```
# # Asignación de variables
a = 5
print(a)
```

```
5
```

[8]:
```
a = 1 # entero
b = 4.0 # coma flotante
c = "ITQ" # string
d = 10 + 1j # numero complejo
e = True #False # boolean
f = None # None
# visualizar valor de las variables y su tipo
print(a)
print(type(a))
print(b)
print(type(b))
print(c)
print(type(c))
print(d)
print(type(d))
print(e)
print(type(e))
print(f)
print(type(f))
```

```
1
<class 'int'>
4.0
<class 'float'>
ITQ
<class 'str'>
(10+1j)
<class 'complex'>
True
<class 'bool'>
None
<class 'NoneType'>
```

- Las variables no tienen tipo.
- Las variables apuntan a objetos que sí lo tienen.
- Dado que Python es un lenguaje de tipado dinámico, la misma variable puede apuntar, en momentos diferentes de la ejecución del programa, a objetos de diferente tipo.

[9]:
```
a = 3
print(a)
print(type(a))
a = 'Pablo García'
print(a)
print(type(a))
a = 4.5
print(a)
print(type(a))
```

```
3
<class 'int'>
Pablo García
<class 'str'>
4.5
<class 'float'>
```

- Garbage collection: Cuando un objeto deja de estar referenciado, se elimina automáticamente.

Identificadores

- Podemos obtener un identificador único para los objetos referenciados por variables.
- Este identificador se obtiene a partir de la dirección de memoria.

[10]:
```
a = 3
print(id(a))
a = 'Pablo García'
print(id(a))
a = 4.5
print(id(a))
```

```
140709578253288
2804688129280
2804686142416
```

- Referencias compartidas: un mismo objeto puede ser referenciado por más de una variable.
  – Variables que referencian al mismo objeto tienen mismo identificador.

[11]:
```
a = 4567
print(id(a))
```

```
2804732909712
```

[12]:
```
b = a
print(id(b))
```

```
2804732909712
```

[13]:
```
c = 4567
print(id(c))
```

```
2804732912304
```

[14]:
```
a = 25
b = 25

print(id(a))
print(id(b))
print(id(25))
```

```
140709578253992
140709578253992
140709578253992
```

[15]:
```
# # Ojo con los enteros "grandes" [-5, 256]
a = 258
b = 258

print(id(a))
print(id(b))
print(id(258))
```

```
2804732910512
2804732907760
2804732909008
```

- Referencia al mismo objeto a través de asignar una variable a otra.

[16]:
```
a = 400
b = a
print(id(a))
print(id(b))
```

```
2804732911600
2804732911600
```

- Las variables pueden aparecer en expresiones.

[17]:
```
a = 3
b = 5
print(id(a))
print (a + b)
```

```
140709578253288
8
```

Respecto a los nombres de las variables ...

- No se puede poner números delante del nombre de las variables.
- Por convención, evitar CamelCase. Mejor usar snake_case: uso de "_" para separar palabras.
- El lenguaje diferencia entre mayúsculas y minúsculas.
- Deben ser descriptivos.
- Hay palabras o métodos reservados -> Built-ins y KeyWords
  – Ojo con reasignar un nombre reservado!

[18]:
```
potencia = 1
print(pow(3, 2))
```

```
9
```

[19]:
```
print(pow(3,2))

pow = 1 # built-in reasignado
print(pow)

print(pow(3,2))
```

```
9
1
```

Fix Code

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[19], line 6
      3 pow = 1 # built-in reasignado
      4 print(pow)
----> 6 print(pow(3,2))

TypeError: 'int' object is not callable
```

[ ]:
```
def pow(a, b):
return a + b
print(pow(3,2))
```

Asignación múltiple de variables

[ ]:
```
x, y, z = 1, 2, 3
print(x, y, z)
t = x, y, z, 7, "Python"
print(t)
print(type(t))
```

- Esta técnica tiene un uso interesante: el intercambio de valores entre dos variables.

[ ]:
```
a = 1
b = 2

a, b = b, a
print(a, b)
```

[ ]:
```
a = 1
b = 2

c = a
a = b
b = c
print(a, b, c)
```

Tipos de datos básicos

Bool

• 2 posibles valores: 'True' o 'False'

[ ]:
```
a = False
b = True

print(a)
print(type(a))

print(b)
print(type(b))
```

- 'True' y 'False' también son objetos que se guardan en caché, al igual que los enteros pequeños.

[ ]:
```
a = True
b = False

print(id(a))
print(id(b))

print(a is b)
print(a == b)
```

Números

[ ]:
```
print(2) # Enteros, sin parte fraccional.
print(3.4) # Números en coma flotante, con parte fraccional.
print(2+4j) # Números complejos.
print(1/2) # Numeros racionales.
```

- Diferentes representaciones: base 10, 2, 8, 16.

[ ]:
```
x = 58 # decimal
z = 0b00111010 # binario
w = 0o72 # octal
y = 0x3A # hexadecimal

print(x == y == z == w)
```

Strings

- Cadenas de caracteres.
- Son secuencias: la posición de los caracteres es importante.
- Son immutables: las operaciones sobre strings no cambian el string original.

[ ]:
```
s = 'John "ee" Doe'
print(s[0]) # Primer carácter del string.
print(s[-1]) # Último carácter del string.
print(s[1:8:2]) # Substring desde el segundo carácter (inclusive) hasta el octavo (exclusive). Esta técnica se la cor
print(s[:]) # Todo el string.
print(s + "e") # Concatenación.
```

Conversión entre tipos

- A veces queremos que un objeto sea de un tipo específico.
- Podemos obtener objetos de un tipo a partir de objetos de un tipo diferente (casting).

[ ]:
```
a = int(2.8) # a será 2
b = int("3") # b será 3
c = float(1) # c será 1.0
d = float("3") # d será 3.0
e = str(2) # e será '2'
f = str(3.0) # f será '3.0'
g = bool("a") # g será True
h = bool("") # h será False
```

```
i = bool(3) # i será True
j = bool(0) # j será False
k = bool(None)

print(a)
print(type(a))
print(b)
print(type(b))
print(c)
print(type(c))
print(d)
print(type(d))
print(e)
print(type(e))
print(f)
print(type(f))
print(g)
print(type(g))
print(h)
print(type(h))
print(i)
print(type(i))
print(j)
print(type(j))
print(k)
```

[ ]:
```
print(7/4) # División convencional. Resultado de tipo 'float'
print(7//4) # División entera. Resultado de tipo 'int'
print(int(7/4)) # Divisón convencional. Conversión del resultado de 'float' a 'int'
```

Operadores

Python3 precedencia en operaciones

- Combinación de valores, variables y operadores
- Operadores y operandos

Operatores aritméticos

Operador	Desc
a + b	Suma
a - b	Resta
a / b	División
a // b	División Entera
a % b	Modulo / Resto
a * b	Multiplicacion
a ** b	Exponenciación

[ ]:
```
x = 3
y = 2
print('x + y = ', x + y)
print('x - y = ', x - y)
print('x * y = ', x * y)
print('x / y = ', x / y)
print('x // y = ', x // y)
print('x % y = ', x % y)
print('x ** y = ', x ** y)
```

Operadores de comparación

Operador	Desc
a > b	Mayor
a < b	Menor

a < b	Menor
a == b	Igualdad
a != b	Desigualdad
a >= b	Mayor o Igual
a <= b	Menor o Igual

```
x = 10 •••
```

Operadores Lógicos

Operador	Desc
a and b	True, si ambos son True
a or b	True, si alguno de los dos es True
a ^ b	XOR - True, si solo uno de los dos es True
not a	Negación

Enlace a Tablas de Verdad.

[ ]:
```
x = True
y = False

print('x and y es :', x and y)
print('x or y es :', x or y)
print('x xor y es :', x ^ y)
print('not x es :', not x)
```

Operadores Bitwise / Binarios

Operador	Desc
a & b	And binario
a ! b	Or binario
a ^ b	Xor binario
~ a	Not binario
a » b	Desplazamiento binario a derecha
a « b	Desplazamiento binario a izquierda

[ ]:
```
# x = 0b01100110
# y = 0b00110011
# print("Not x = " + bin(~x))
# print("x and y = " + bin(x & y))
# print("x or y = " + bin(x | y))
# print("x xor y = " + bin(x ^ y))
# print("x << 2 = " + bin(x << 2))
# print("x >> 2 = " + bin(x >> 2))
```

Operadores de Asignación

Operador	Desc
=	Asignación
+=	Suma y asignación
-=	Resta y asignación
*=	Multiplicación y asignación
/=	División y asignación
%=	Módulo y asignación
//=	División entera y asignación
**=	Exponencial y asignación

&=	And y asignación
!=	Or y asignación
^=	Xor y asignación
»=	Despl. Derecha y asignación
«=	DEspl. Izquierda y asignación

[ ]:
```
a = 5
a *= 3 # a = a * 3
a += 1 # No existe a++, ni ++a,a--,--a
print(a)

b = 6
b -= 2 # b = b - 2
print(b)
```

Operadores de Identidad

Operador	Desc
a is b	True, si ambos operadores son una referencia al mismo objeto
a is not b	True, si ambos operadores no son una referencia al mismo objeto

[ ]:
```
a = 4444
b = a
print(a is b)
print(a is not b)
```

Operadores de Pertenencia

Operador	Desc
a in b	True, si a se encuentra en la secuencia b
a not in b	True, si a no se encuentra en la secuencia b

[ ]:
```
x = 'Hola Mundo'
y = {1:'a',2:'b'}

print('H' in x) # True
print('hola' not in x) # True

print(1 in y) # True
print('a' in y) # False
```

Entrada de valores

[ ]:
```
valor = input("Inserte valor:")
print(valor)
print(type(valor))
```

[ ]:
```
grados_c = int(input("Conversión de grados a fahrenheit, inserte un valor: "))
print(f"Grados F: {1.8 * (grados_c) + 32}")
```

Elaborar un programa que permita calcular el volumen de un cilindro, solicitando al usuario que ingrese los valores por teclado: radio y altura del cilindro. El resultado debe mostrarse en pantalla, No se debe utilizar condicionales, ni funciones matemáticas, solo operaciones aritméticas básicas y lectura de datos por teclado. Fórmula del volumen:

$$V = pi * r^2 * h$$

r= radio h= altura que se ingrese por teclado

[ ]:
```
#Solución

# Programa para calcular el volumen de un cilindro

pi = 3.1416
```

```
r = float(input("Ingrese el radio del cilindro: "))
h = float(input("Ingrese la altura del cilindro: "))

V = pi * r * r * h

print("El volumen del cilindro es:", V)
```

Elaborar un programa que permita convertir una distancia medida en metros a centimetros, milimetros y kilometros. El usuaario debe ingresar el valor en metros y el programa debe mostrar las tres equivalencias correspondientes. No se debe utilizar condicionales ni funciones matemáticas, únicamente operaciones aritméticas básicas.

[ ]:
```
#Solución

# Programa para convertir metros a centimetros, milimetros y kilometros

m = float(input("Ingrese la distancia en metros: "))

cm = m * 100
mm = m * 1000
km = m / 1000

print("Distancia en centimetros:", cm, "cm")
print("Distancia en milimetros:", mm, "mm")
print("Distancia en kilometros:", km, "km")
```

3 Tipos de datos compuestos (colecciones)

3.1 Listas

- Una colección de objetos.
- Mutables.
- Tipos arbitrarios heterogeneos.
- Puede contener duplicados.
- Notienen tamaño fijo. Pueden contener tantos elementos como quepan en memoria.
- Los elementos van ordenados por posición.
- Se acceden usando la sintaxis: [index].
- Los índices van de 0 a n-1, donde n es el número de elementos de la lista.
- Son un tipo de Secuencia, al igual que los strings; por lo tanto, el orden (es decir, la posición de los objetos de la lista) es importante.
- Soportan anidamiento.
- Son una implementación del tipo abstracto de datos: Array Dinámico.

Operaciones con listas

- Creación de listas.

[ ]:
```
letras = ['a', 'b', 'c', 'd']
palabras = 'Hola mundo como estas'.split()
numeros = list(range(7))
print(letras)
print(palabras)
print(numeros)
print(type(numeros))
```

[ ]:
```
# # Pueden contener elementos arbitrarios / heterogeneos
mezcla = [1, 3.4, 'a', None, False]
print(mezcla)
print(len(mezcla)) # len me da el tamaño de la lista número de elementos
```

[ ]:
```
# # Pueden incluso contener objetos más "complejos"
lista_con_funcion = [1, 2, len, pow]
print(lista_con_funcion)
```

[ ]:
```
# # Pueden contener duplicados
lista_con_duplicados = [1, 2, 3, 3, 3, 4]
print(lista_con_duplicados)
```

- Obtención de la longitud de una lista.

[ ]:
```
letras = ['a', 'b', 'c', 'd', 1]
print(len(letras))
```

- Acceso a un elemento de una lista

[ ]:
```
print(letras[2])
print(letras[-5])
print(letras[0])
```

- Slicing: obtención de un fragmento de una lista, devuelve una copia de una parte de la lista
  – Sintaxis: *lista [inicio : fin : paso]*

[ ]:
```
letras = ['a', 'b', 'c', 'd', 'e']

print(letras[1:3])
print(letras[:3])
print(letras[:-1])
print(letras[2:])
print(letras[:])
print(letras[::2])
```

[ ]:
```
letras = ['a', 'b', 'c', 'd']

print(letras)
print(id(letras))

a = letras[:]
print(a)
print(id(a))

print(letras.copy())
print(id(letras.copy()))
```

- Añadir un elemento al final de la lista

[ ]:
```
letras.append('e')
print(letras)
print(id(letras))
```

[ ]:
```
letras += 'e'
print(letras)
print(id(letras))
```

- Insertar en posición.

[ ]:
```
print(len(letras))
letras.insert(1,'g')
print(len(letras))
print(letras)
print(id(letras))
```

- Modificación de la lista (individual).

[ ]:
```
letras[5] = 'f'
print(letras)
print(id(letras))
```

[ ]:
```
# # index tiene que estar en rango
letras[20] = 'r'
```

[ ]:
```
* Modificación múltiple usando slicing.
```

[ ]:
```
letras = ['a', 'b', 'c', 'f', 'g', 'h', 'i', 'j', 'k']
print(id(letras))
```

[ ]:
```
letras[0:7:2] = ['z', 'x', 'y','p']
print(letras)
```

```
print(letras)
# print(id(letras))
```

[ ]:
```
# # Ojo con la diferencia entre modificación individual y múltiple. Asignación
individual de lista crea anidamiento.
numeros = [1, 2, 3]
numeros[1] = [10, 20, 30]
print(numeros)
print(numeros[1][0])
numeros[1][2] = [100, 200]
print(numeros)
print(numeros[1][2][1])
```

- Eliminar un elemento.

[ ]:
```
#letras.remove('f')
if 'p' in letras:
letras.remove('p')
#
#letras.remove('z')
print(letras)
```

[ ]:
```
# elimina el elemento en posición-1 y lo devuelve
elemento = letras.pop()
print(elemento)
print(letras)
```

[ ]:
```
numeros = [1, 2, 3]
print(numeros)
numeros[2] = [10, 20, 30]
print(numeros)
n = numeros[2].pop()
print(numeros)
print(n)
```

[ ]:
```
# numeros1=10
# print(numeros1)
# print(numeros)
```

[ ]:
```
lista = []
a = lista.pop()
```

- Encontrar índice de un elemento.

[ ]:
```
letras = ['a', 'b', 'c', 'c']
if 'a' in letras:
print(letras.index('a'))
print(letras)
```

- Concatenar listas.

[ ]:
```
lacteos = ['queso', 'leche']
frutas = ['naranja', 'manzana']
print(id(lacteos))
print(id(frutas))
compra = lacteos + frutas
print(id(compra))
print(compra)
```

[ ]:
```
# # Concatenación sin crear una nueva lista
frutas = ['naranja', 'manzana']
print(id(frutas))
frutas.extend(['pera', 'uvas'])
print(frutas)
print(id(frutas))
```

[ ]:
```
# # Anidar sin crear una nueva lista
frutas = ['naranja', 'manzana']
print(id(frutas))
frutas.append(['pera', 'uvas'])
print(frutas)
print(id(frutas))
```

- Replicar una lista.

[ ]:
```
lacteos = ['queso', 'leche']
print(lacteos * 3)
print(id(lacteos))

a = 3 * lacteos
print(a)
print(id(a))
```

- Copiar una lista

[ ]:
```
frutas2 = frutas.copy()
frutas2 = frutas[:]
print(frutas2)
print('id frutas = ' + str(id(frutas)))
print('id frutas2 = ' + str(id(frutas2)))
```

- Ordenar una lista.

[ ]:
```
lista = [4,3,8,1]
print(lista)
lista.sort()
print(lista)

lista.sort(reverse=True)
print(lista)
```

[ ]:
```
# # Los elementos deben ser comparables para poderse ordenar
lista = [1, 'a']
lista.sort()
```

[ ]:
```
compra = ['Huevos','Pan','zapallo','Leche','Licor']
print(sorted(compra))
print(compra)
```

- Pertenencia

[ ]:
```
lista = [1, 2, 3, 4]
print(1 in lista)
print(5 in lista)
```

- Anidamiento.

[ ]:
```
letras = ['a', 'b', 'c', ['x', 'y', ['i', 'j', 'k']]]
print(letras[0])
print(letras[3][0])
print(letras[3][2][0])
```

[ ]:
```
print(letras[3])
```

[ ]:
```
a =[1000,2,3]
b = a[:] #a.copy() #[:]
print(id(a))
print(id(b))
print(id(a[0]))
print(id(b[0]))
```

Alias/Referencias en las listas

- Son mutables y las asignaciones a un objeto alteran el primero
- Pasar listas a funciones puede suponer un riesgo
- Clonar o copiar listas

[ ]:
```
lista = [2, 4, 16, 32]
ref = lista

print(id(lista))
```

```
print(id(ref))

ref[2] = 64

print(ref)
print(lista)
```

[ ]:
```
lista = [2000, 4, 16, 32]
copia = lista[:]
copia = lista.copy()

copia[2] = 64

print(lista)
print(copia)

print(id(lista))
print(id(copia))

print(id(lista[2]))
print(id(copia[2]))
```

[ ]:
```
# #listas anidadas se copian por referencia
lista = [2, 4, 16, 32, [34, 10, [5,5]]]
copia = lista.copy()

copia[3] = 20
copia[4][0] = 28

print(copia)
print(lista)
```

[ ]:
```
# lista = [0, 1, [10, 20]]
# print(id(lista))

# lista2 = [10, 20]
# lista = [0, 1, lista2]

# copia = lista[:] #.copy()
# print(copia)
# print(id(copia))

# print(id(lista[2]))

# print(id(copia[2]))

# copia[2][0] = 40
# print(copia)
# print(lista)
```

[ ]:
```
# # evitar que listas anidadas se copien por referencia
# import copy

# lista = [2, 4, 16, 32, [34, 10, [5,5]]]

# copia = copy.deepcopy(lista)
# copia[0] = 454
# copia[4][2][0] = 64
# print(lista)
# print(copia)

# print(f"{id(lista)}- {id(copia)}")
# print(f"{id(lista[4])}- {id(copia[4])}")
```

3.2 Diccionarios

- Colección de parejas clave-valor.
- Son mutables.
- Claves:
  – Cualquier objeto inmutable.
  – Sólo pueden aparecer una vez en el diccionario.
- Valores:
  – Sin restricciones. Cualquier objeto (enteros, strings, listas, etc.) puede hacer de valor.

- Desde la versión 3.7 están ordenados.
- Se pueden ver como listas indexadas por cualquier objeto inmutable, no necesariamente por números enteros.
- Adiferencia de las listas, no son secuencias. Son mappings.

Operaciones con diccionarios

- Creación de diccionarios.

[ ]:
```
 # Creación simple, usando una expresión literal.
persona = {'DNI' : '11111111D', 'Nombre' : 'Carlos', 'Edad' : 34}
print(persona)
```

[ ]:
```
# Creación uniendo dos colecciones.

nombres = ['Pablo', 'Manolo', 'Pepe', 'Juan']
edades = [52, 14, 65]
datos = dict(zip(nombres,edades))
print(datos)
```

[ ]:
```
# Creación pasando claves y valores a la función 'dict'
persona2 = dict(nombre='Rosa', apellido='Garcia')
print(persona2)
```

[ ]:
```
# Creación usando una lista de tuplas de dos elementos.

persona2 = dict([('nombre', 'Rosa'), ('apellido','Garcia')])
print(persona2)
```

[ ]:
```
# Creación incremental por medio de asignación (como las claves no existen, se crean nuevos items)

persona = {}
print(persona['DNI'])
persona['DNI'] = '11111111D'
persona['Nombre'] = 'Carlos'
persona['Edad'] = 34
persona['DNI'] = '222222222'
print(persona)
```

- Acceso a un valor a través de la clave.

[ ]:
```
persona = {'DNI' : '11111111D', 'Nombre' : 'Carlos', 'Edad' : 34}
print(persona['Nombre'])
```

[ ]:
```
# Acceso a claves inexistentes o por índice produce error

# persona[1]
print(persona['Nombre'])

if 'Trabajo' in persona:
    print(persona['Trabajo'])
else:
    print("En el paro")

persona.get('Trabajo', "En el paro")
```

- Modificación de un valor a través de la clave.

[ ]:
```
persona = {'DNI' : '11111111D', 'Nombre' : 'Carlos', 'Edad' : 34}

persona['Nombre'] = 'Fernando'
persona['Edad'] += 1

print(persona)
```

- Añadir un valor a través de la clave.

[ ]:
```
persona = {'DNI' : '11111111D', 'Nombre' : 'Carlos', 'Edad' : 34}
persona['Ciudad'] = 'Valencia'
print(persona)
```

- Eliminación de un valor a través de la clave.

[ ]:
```
del persona['Ciudad']
value = persona.pop('Edad')

print(persona)
print(value)
```

- Comprobación de existencia de clave.

[ ]:
```
print('Nombre' in persona)
print('Apellido' in persona)
```

- Recuperación del valor de una clave, indicando valor por defecto en caso de ausencia.

[ ]:
```
persona = {'Nombre' : 'Carlos'}

value = persona.get('Nombre')
print(value)

# persona['Estatura']

value = persona.get('Estatura', 180)
print(value)
```

- Anidamiento.

[ ]:
```
persona = {
'Trabajos' : ['desarrollador', 'gestor'],
'Direccion' : {'Calle' : 'Pintor Sorolla', 'Ciudad' : 'Valencia'}
}

print(persona['Direccion'])
print(persona['Direccion']['Ciudad'])
```

- Métodos items, keys y values.

[ ]:
```
persona = {'DNI' : '11111111D', 'Nombre' : 'Carlos', 'Edad' : 34}

print(list(persona.items()))
print(list(persona.keys()))

print(list(persona.values()))
```

[ ]:
```
dict_simple = {'ID' : 'XCSDe1194', 'Nombre' : 'Carlos', 'Edad' : 34}
# iterar sobre las claves
print('Claves\n')
for key in dict_simple.keys(): # o dict_simple
print(key)
print('\nValores\n')
# iterar sobre los valores
for value in dict_simple.values():
print(value)
```

[ ]:
```
dict_simple = {'ID' : 'XCSDe1194', 'Nombre' : 'Carlos', 'Edad' : 34}
# iterar sobre ambos, directamente con items().
for key, value in dict_simple.items():
print('Clave: ' + str(key) + ', valor: ' + str(value))

for item in dict_simple.items(): # devuelve tuplas
print('Item: ' + str(item))

# usar zip para iterar sobre Clave-Valor
for key, value in zip(dict_simple.keys(),dict_simple.values()):
print('Clave: ' + str(key) + ', valor: ' + str(value))
```

Sets (Conjuntos)

Al igual que las listas:

- Colección de elementos

- Colección de elementos.
- Tipos arbitrarios.
- Mutables.
- Notienen tamaño fijo. Pueden contener tantos elementos como quepan en memoria.
  A diferencia de las listas:
- No puede tener duplicados.
- Se definen por medio de llaves.
- Los elementos no van ordenados por posición. No hay orden establecido.
- Solo pueden contener objetos inmutables.
- Nosoportan anidamiento.

Operaciones con conjuntos

- Creación de conjuntos.

[ ]:
```
set1 = {0, 1, 1, 2, 3, 4, 4}
print(set1)

set2 = {'user1', 12, 2}
print(set2)

set3 = set(range(7))
print(set3)

set4 = set([0, 1, 2, 3, 4, 0, 1])
print(set4)
```

[ ]:
```
#Observa la diferencia entre listas y conjuntos
s = 'aabbc'
print(list(s))
print(set(s))
```

- Acceso por índice genera error.

[ ]:
```
set1 = {0, 1, 2}
print(set1[0])
```

- Unión, intersección y diferencia.

[ ]:
```
set1 = {0, 1, 1, 2, 3, 4, 5, 8, 13, 21}
set2 = set([0, 1, 2, 3, 4, 42])

# union
print(set1 | set2)

# intersección
print(set1 & set2)

# diferencia
print(set1- set2)
print(set2- set1)
```

[ ]:
```
a = [1,2]
b = [2,3]
print(set(a) & set(b))
```

[ ]:
```
#Además de los operadores, que operan únicamente con Sets, también se pueden usar métodos que pueden operar sobre cuc
conjunto = {0, 1, 2}
lista = [1, 3, 3]

print(conjunto.union(lista))
print(conjunto.intersection(lista))
print(conjunto.difference(lista))
```

- comparación de conjuntos.

[ ]:
```
set1 = {0, 1, 1, 2, 3, 4, 5, 8, 13, 21}
set2 = set([0, 1, 2, 3, 4])
```

```
print(set2.issubset(set1))
print(set1.issuperset(set2))
print(set1.isdisjoint(set2))
```

- Pertenencia.

[ ]:
```
words = {'calm', 'balm'}
print('calm' in words)
```

- Anidamiento.

[23]:
```
# Los conjuntos no soportan anidamiento, pero como permite elementos inmutables, se pueden "anidar" tuplas.
nested_set = {1, (1, 1, 1), 2, 3, (1,1,1)}
print(nested_set)
```

```
{1, 2, 3, (1, 1, 1)}
```

- Modificación de conjuntos.

[20]:
```
# A través de operador de asignación

set1 = {'a', 'b', 'c'}
set2 = {'a', 'd'}

# set1 |= set2 # set1 = set1 | set2
# set1 &= set2
set1-= set2

print(set1)
```

```
{'c', 'b'}
```

[21]:
```
# A través de método 'update'.

set1 = {'a', 'b', 'c'}
set2 = {'a', 'd'}

# set1.update(set2)
# set1.intersection_update(set2)
set1.difference_update(set2)

print(set1)
```

```
{'c', 'b'}
```

[22]:
```
# A través de métodos 'add' y 'remove'.

set1 = {'a', 'b', 'c'}

set1.add('d')

set1.remove('a')

print(set1)
```

```
{'d', 'c', 'b'}
```

[ ]:

Click to add a cell.